

تقرير وظيفة البرمجة التفرعية

إعداد:

ورد عدنان اختيار – 43396 محمد أحمد ياسر صيرفي - 43294
محمد عادل المحمد – 43285 محمد ملاذ مزيد احمد – 43332
محمد غيث محمد جمال شبقجي – 43227

1. حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

الشرح :

- يتطلب هذا البند حل مشكلة "حالة السباق (Race Condition)" ، وهي الظاهرة البرمجية التي تحدث عندما تحاول عدة خيوط (Threads) أو عمليات (Processes) تعديل نفس المورد المشترك في قاعدة البيانات (مثل كمية مخزون منتج معين أو سلة تسوق) في نفس الجزء من الثانية.
- إذا لم يتم ضبط الوصول، ستتداخل العمليات ويحفظ تضارب في البيانات (Data Corruption)، كأن يتم بيع منتج لم يعد متوفرا في المخزن.

تطبيقه في المشروع :

- قمت بحل هذه المشكلة عبر تطبيق **قفل الكتابة التشاؤمي (Pessimistic Write Lock)** على مستوى السطر (Row-level Lock) في قاعدة البيانات حيث يقوم النظام بحجز وقفل السطر الذي يتم التعديل عليه لصالح العملية الأولى الحالية، مما يجبر بقية العمليات المترامنة على الانتظار في طابور قاعدة البيانات حتى تنتهي المعاملة (Transaction) الأولى وتقوم بالحفظ (Save)، وبذلك يتم حظر التعديل المتداخل نهائيا وضمان سلامة البيانات.

2. إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

الشرح:

- يركز هذا المتطلب على إيجاد توازن مثالي في استهلاك موارد السيرفر (المعالج CPU والذاكرة RAM). إذا ترك النظام لاستقبال ومعالجة عدد غير محدود من العمليات الحسابية المتوازية والمعقدة في نفس اللحظة، سيؤدي ذلك إلى استهلاك مفرط للموارد وبالتالي انهيار الخادم (Server Crash). وفي المقابل، إذا تم تقييد العمليات المتوازية بشكل مفرط، سيصبح النظام بطيئا جدا في الاستجابة للمستخدمين.

تطبيقه في المشروع:

- يتم التحكم في هذا المتطلب عبر ضبط عدد الـ **Workers** والمهام المتوازية (Concurrency Limit) في **BullMQ** حيث يقوم النظام بتحديد سقف أعلى لعدد المهام التي يعالجها السيرفر في الخلفية بالتوازي بناء على قدرة أنوية المعالج، مما يمنع تجاوز الحد الأقصى للموارد ويضمن بقاء النظام مستقرا وسريع الاستجابة دون خطر الانهيار.

3. المعالجة غير المتزامنة (Asynchronous Queues)

الشرح :

- يهدف هذا المتطلب إلى فصل "مسار الطلب الرئيسي (Request-Response Cycle)" عن المهام الثانوية الثقيلة التي لا يحتاج المستخدم لانتظار نتائجها فوراً (مثل إرسال رسائل التأكيد، أو الإشعارات، أو توليد الفواتير). (إجبار المستخدم على انتظار هذه العمليات يؤدي إلى زيادة زمن الاستجابة وتجربة مستخدم سيئة).

تطبيقه في المشروع:

- قمت بتطبيق هذا الفصل باحترافية عن طريق دمج BullMQ المدعوم ب Redis فعندما يقوم المستخدم بطلب إضافة إلى السلة أو إتمام الدفع، يستقبل السيرفر الطلب ويلقيه فوراً كـ "مهمة خلفية (Job)" داخل Redis، ويرد فوراً على المستخدم برمز الاستجابة Accepted 202 ليتصفح الموقع بحرية، بينما يتولى BullMQ معالجة تلك المهام الثقيلة في الخلفية بشكل منفصل تماماً عن مسار خيط النظام الرئيسي.

4. معالجة البيانات الضخمة على دفعات (Batch Processing)

الشرح :

- عند التعامل مع كميات ضخمة من البيانات (مثل جرد المبيعات اليومية أو تحديث مخزون آلاف المنتجات دفعة واحدة)، فإن معالجة كل سجل على حدة تستهلك اتصالات شبكة كثيفة ومجهدة لقاعدة البيانات. يتطلب هذا البند تجميع هذه البيانات ومعالجتها على شكل مجموعات أو دفعات مقننة ومقسمة (Chunks / Patches) لتقليل العبء وتسريع الأداء الإجمالي.

تطبيقه في المشروع :

- تم تطبيق ذلك برمجياً من خلال آليات التجزئة في BullMQ حيث يتم تقسيم العمليات الضخمة القادمة إلى دفعات برمجية (Patches)، ويقوم الـ Worker بمعالجة كل دفعة بشكل مركز ومتواز، مما يحافظ على توازن استهلاك الذاكرة ويمنع تجميد حلقة الأحداث (Event Loop) الخاصة بـ NestJS أثناء معالجة البيانات الضخمة.
-

5. توزيع الأحمال (Load Distribution)

الشرح :

- يعتمد هذا المتطلب على محاكاة أو تطبيق معمارية تمنع تركيز كامل ضغط طلبات المستخدمين المتزامنة على خادم واحد. يتم ذلك من خلال توزيع حركة المرور (Traffic) على أكثر من نسخة من التطبيق (Multiple Instances) بناءً على استراتيجية توزيع واضحة ومبررة تضمن عدم غرق أي خادم فردي تحت الضغط.

تطبيقه في المشروع:

- قمت بدمج خادم **Nginx** كبوابة أمامية وموازن أحمال (Reverse Proxy & Load Balancer) حيث يستقبل Nginx سيل الطلبات الكثيفة الصادرة من اختبارات الضغط، ويقوم بتوزيعها بذكاء بالتساوي على نسخ تطبيق **NestJS** المتعددة المتاحة في البنية التحتية، مما يضمن امتصاص الصدمات وتحقيق توزيع عادل للحمل على مستوى النظام بالكامل لمنع سقوط الخدمة.